Assignment-13.4

2303A510E3

BATCH-14

Task 1: Refactoring Data Transformation Logic

Scenario

You are maintaining a data preprocessing script where numerical

transformations are written using verbose loops.

Task Description

• Review the legacy code that computes transformed values using

an explicit loop.

• Use an AI tool to suggest a more Pythonic refactoring approach.

• Refactor the code using list comprehensions or helper functions

while preserving the output.

Legacy Code

values = [2, 4, 6, 8, 10]

doubled = []

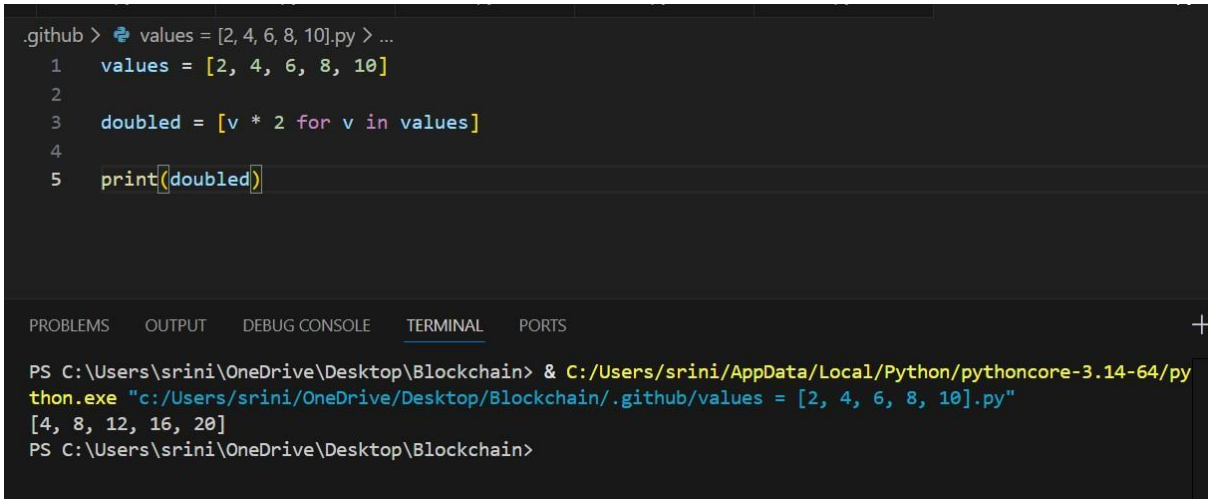
for v in values:

doubled.append(v * 2)

print(doubled)

Expected Output

[4, 8, 12, 16, 20]



Task 2: Improving Text Processing Code Readability

Scenario

You are working on a log message generator that builds messages word

by word.

Task Description

• Analyze the legacy code that constructs a sentence using repeated

string concatenation.

• Ask AI to suggest a more efficient and readable approach.

• Refactor the code accordingly while keeping the final output

unchanged.

Legacy Code

```
words = ["Refactoring", "with", "AI", "improves",

"quality"]

message = ""

for w in words:

message += w + " "

print(message.strip())
```

Expected Output

```
Refactoring with AI improves quality
```

```
.github > JJN.py > ...
words = ["Refactoring", "with", "AI", "improves", "quality"]

message = " ".join(words)

print(message)
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\srini\OneDrive\Desktop\Blockchain>  & 'c:\Users\srini\AppData\Local\Python\pythoncore-3.14-64\
python.exe' 'c:\Users\srini\.vscode\extensions\ms-python.debugpy-2025.19.2026022401-win32-arm64\bundled\li
bs\debugpy\launcher' '55657' '--' 'C:\Users\srini\OneDrive\Desktop\Blockchain\.github\JJN.py'
Refactoring with AI improves quality
```

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may

or may not exist depending on deployment settings.

Task Description

• Review the legacy code that manually checks for dictionary keys.

• Use AI suggestions to refactor the code using safer dictionary

access methods.

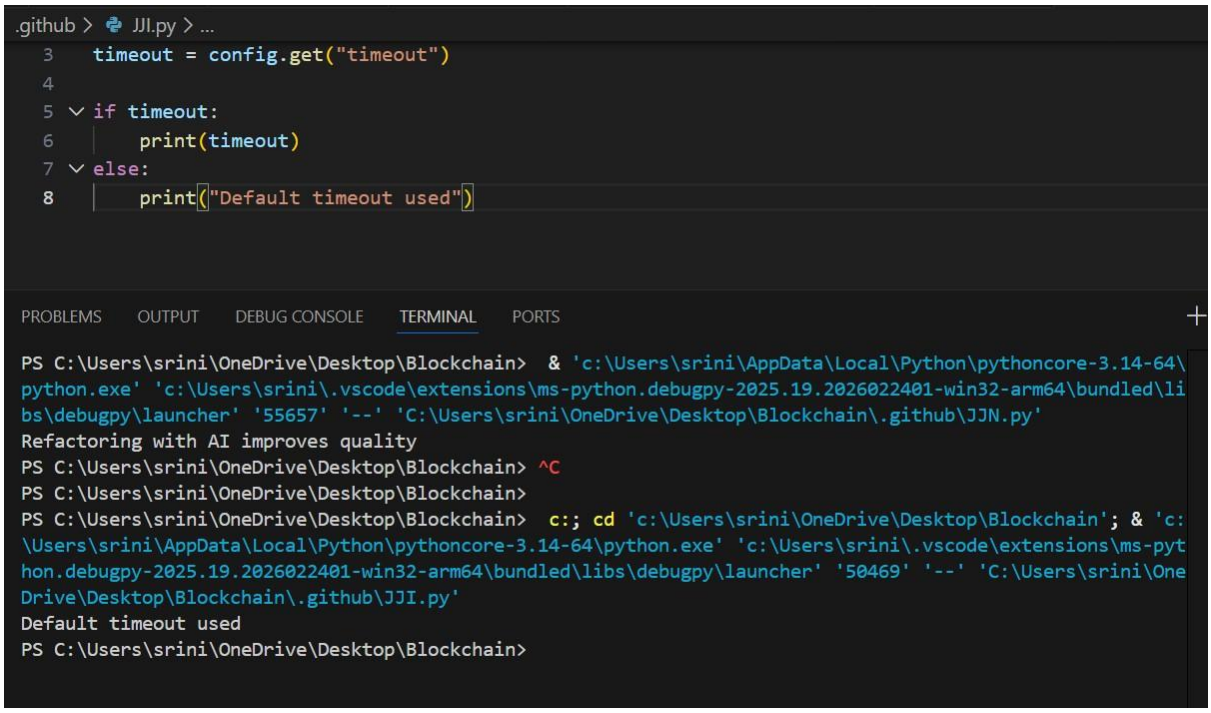
• Ensure the behavior remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
print(config["timeout"])
else:
print("Default timeout used")
```

Expected Output

Default timeout used



Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations

based on user input.

Task Description

• Examine the multiple if–elif conditions used to determine

operations.

• Ask AI to suggest a cleaner, scalable alternative.

• Refactor the logic using mapping techniques while preserving

functionality.

Legacy Code

```
action = "divide"

x, y = 10, 2

if action == "add":

result = x + y

elif action == "subtract":
```

result = x - y

elif action == "multiply":

result = x * y

elif action == "divide":

result = x / y

else:

result = None

print(result)

Expected Output 5.0

.github > FTTF.txt

```python
action = "divide"
x, y = 10, 2

operations = {
    "add": x + y,
    "subtract": x - y,
    "multiply": x * y,
    "divide": x / y
}

result = operations.get(action, None)

print(result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
bs\debugpy\launcher' '55657' '--' 'C:\Users\srini\OneDrive\Desktop\Blockchain\.github\JJN.py'
Refactoring with AI improves quality
PS C:\Users\srini\OneDrive\Desktop\Blockchain> ^C
PS C:\Users\srini\OneDrive\Desktop\Blockchain>
PS C:\Users\srini\OneDrive\Desktop\Blockchain>  c:; cd 'c:\Users\srini\OneDrive\Desktop\Blockchain'; & 'c:
\Users\srini\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\srini\.vscode\extensions\ms-pyt
hon.debugpy-2025.19.2026022401-win32-arm64\bundled\libs\debugpy\launcher' '50469' '--' 'C:\Users\srini\One
Drive\Desktop\Blockchain\.github\JJI.py'
Default timeout used
PS C:\Users\srini\OneDrive\Desktop\Blockchain> ^C
PS C:\Users\srini\OneDrive\Desktop\Blockchain>
PS C:\Users\srini\OneDrive\Desktop\Blockchain>  c:; cd 'c:\Users\srini\OneDrive\Desktop\Blockchain'; & 'c:
\Users\srini\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\srini\.vscode\extensions\ms-pyt
hon.debugpy-2025.19.2026022401-win32-arm64\bundled\libs\debugpy\launcher' '62133' '--' 'C:\Users\srini\One
Drive\Desktop\Blockchain\.github\FTTF.txt'
5.0
PS C:\Users\srini\OneDrive\Desktop\Blockchain>
```

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops

to find elements.

Task Description

• Identify the explicit loop used for searching an item.

• Use AI assistance to refactor the logic into a more concise and

readable form.

• Maintain the same output behavior.

Legacy Code

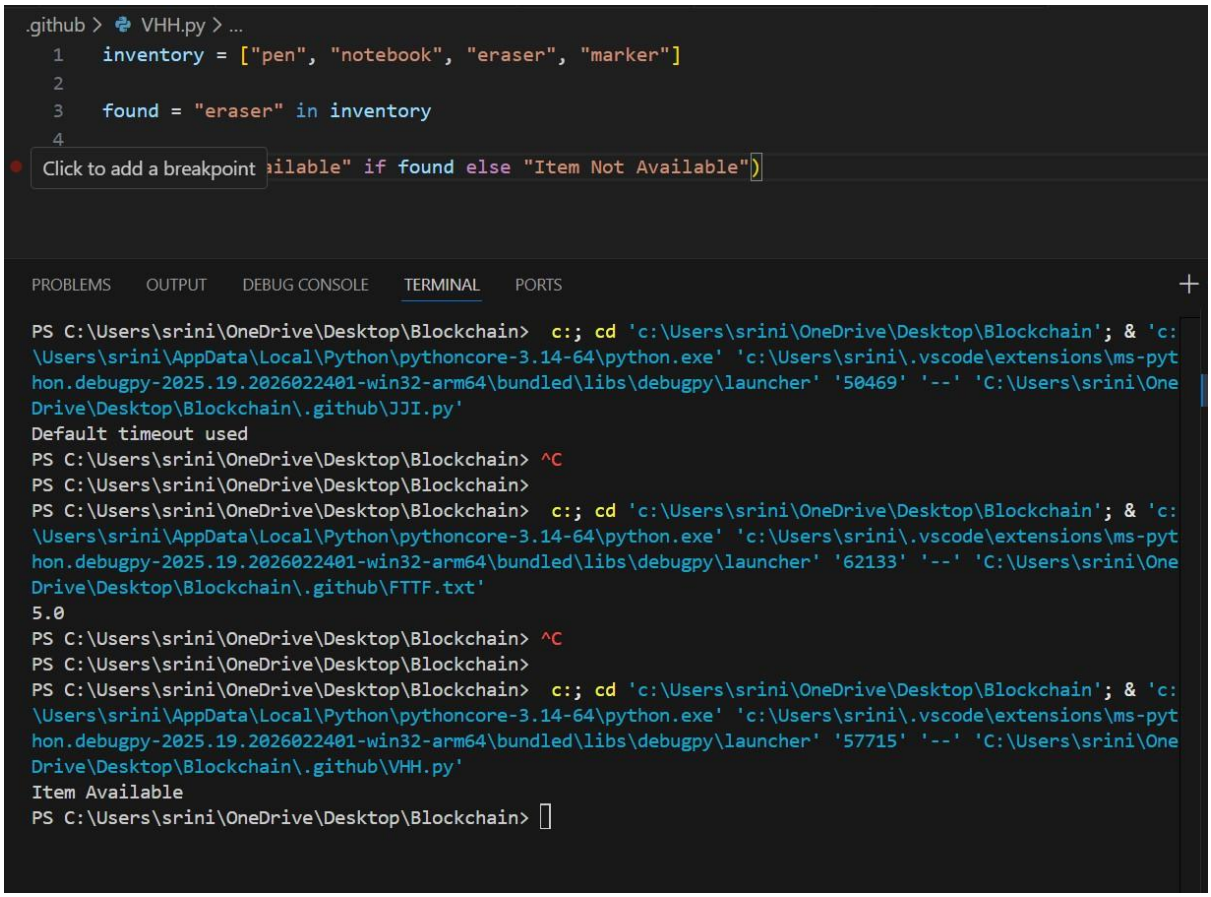
```
inventory = ["pen", "notebook", "eraser", "marker"]

found = False

for item in inventory:
```

if item == "eraser":

found = True

break

print("Item Available" if found else "Item Not Available")

Expected Output

Item Available